Sharpness-Aware Minimization Beyond CNNs: Evaluating Transformers and Generative Models

Group Members: Tom Stanic, Andreas Theilgaard, Théo Basseras
Optimization for Machine Learning, Spring 2025, EPFL.

***Abstract*—Sharpness-Aware Minimization (SAM) is a recent optimization algortihm that promotes better generalization by favoring solutions in flat regions of the loss landscape. While SAM has shown strong performance on CNNs, its effectiveness on transformers and generative models remains underexplored. In this work, we address this gap by comparing SAM to standard optimizers (SGD and Adam) across both classification architectures (ResNet18, ViT) and generative models (score-based diffusion, DDPM, and flow matching). Our experiments highlight scenarios where SAM improves robustness and generalization, in particular for generative models under parameter perturbations.**

Code is available here

I. Introduction

While SGD and Adam [1] are the default optimizers in deep learning, recent work focusing on improving generalization has emerged. One such method is Sharpness-Aware Minimization (SAM) [2], which, unlike standard optimizers, encourages convergence to flat minima where the loss remains low in the surrounding neighborhood. Flatter minima have been associated with improved generalization and robustness which are important in safety-critical domains such healthcare and autonomous systems [3], [4]. While SAM has been extensively studied in CNNs [2], [5], [6], its effectiveness on transformers and generative models is still largely unexplored.

We address this gap by comparing SAM to standard optimizers (SGD and Adam) across both classification architectures (ResNet18 [7], ViT [8]) and generative models (score-based diffusion [9], DDPM [10], and flow matching [11]). Our goal is to assess whether the robustness and generalization benefits observed in CNNs extend to these architectures and models.

II. Background

A. Sharpness-Aware Minimization

Sharpness-Aware Minimization (SAM) [2] reframes optimization as the following minimax problem:

$$\min_{\boldsymbol{w}} \max_{|\varepsilon|_p \leq \rho} \ell(\boldsymbol{w} + \boldsymbol{\varepsilon}), \tag{1}$$

where ℓ is the loss, $\boldsymbol{w}$ are the model parameters, and ρ defines the radius of a neighborhood (typically an L_2-ball, i.e., $p = 2$) around $\boldsymbol{w}$. This formulation encourages solutions where the loss remains low across nearby perturbations, promoting convergence to flatter minima, which are empirically linked to better generalization [2].

SAM is implemented as a wrapper around standard optimizers like SGD or Adam. Instead of minimizing the loss at the current parameters $\boldsymbol{w}_t$, SAM perturbs $\boldsymbol{w}_t$ in the direction of steepest ascent and evaluates the gradient at this perturbed point. The inner maximization of Eq. (1) is approximated via a first-order Taylor expansion of the loss with respect to $\boldsymbol{\varepsilon}$ around zero, yielding the following SAM update rule

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t \nabla \ell(\boldsymbol{w}_t + \hat{\boldsymbol{\varepsilon}}), \tag{2}$$

where $\hat{\boldsymbol{\varepsilon}} = \rho \frac{\nabla \ell(\boldsymbol{w}_t)}{\|\nabla \ell(\boldsymbol{w}_t)\|_2}$ and η_t is the learning rate. This procedure penalizes sharp directions in the loss landscape, encouraging updates that favor robust flat minima.

III. Experimental Setup

In this section we describe the setup for our experiments. We consider both classification and generative tasks. To ensure comparability, all experiments use the same random seed for model initialization and data loading.

A. Classification Setup

We evaluate classification performance on MNIST [12] and CIFAR-10 [13], both containing 10 classes. The models include ResNet18 [7] and a small Vision Transformer (ViT) [8]. Each model is trained using cross-entropy with SGD or Adam [1], as well as their SAM variants using $\rho \in \{0.05, 0.15, 0.3\}$. We use a batch size of 256 and base learning rates of 0.01 for SGD and 0.001 for Adam, except for ViT with Adam, where the learning rate is scaled by 0.1. All models are trained for 100 epochs on CIFAR-10 and 10 epochs on MNIST using an NVIDIA V100 GPU. See Table III and Table IV for full hyperparameter details.

To evaluate robustness, we introduce label noise by randomly flipping labels with probability 0.1 or 0.2. We hypothesize that SAM, by encouraging flatter minima, is more resilient under noisy training. We also assess robustness under adversarial attack using FFGSM [14], which serves as another proxy for robustness. Finally, we quantify sharpness by analyzing the loss curvature at the final model parameters. We visualize the loss surface along the top two hessian eigenvectors and estimate the full eigenvalue spectrum density using the stochastic Lanczos algorithm [15], [16], providing insight into both worst-case and overall curvature.

B. Generative Models Setup

The generative models are parameterized by a U-Net transformer architecture [10], [17] and trained on CIFAR10. We consider a score-based diffusion model [9], a denoising diffusion probabilistic model (DDPM) [10], and a flow matching model [11], each trained with Adam and SAM (using Adam as the base optimizer). Due to computational constraints, we use a single SAM setting with $\rho = 0.05$. The

Dataset	Model	Optimizer	Adversarial Accuracy (%)			Accuracy (%)		
			P=0	P=0.1	P=0.2	P=0	P=0.1	P=0.2
CIFAR-10	ResNet18	adam	19.0	**25.3**	24.7	85.2	**86.8**	**87.0**
	ResNet18	sam-adam	**22.1**	25.2	**27.0**	**85.6**	86.7	86.8
	ResNet18	sam-sgd	**18.5**	**20.4**	**22.4**	**78.4**	**79.8**	**80.3**
	ResNet18	sgd	8.2	10.8	12.6	71.5	72.6	72.2
	ViT	adam	**10.4**	**10.5**	11.7	72.5	71.3	71.5
	ViT	sam-adam	9.6	10.3	**12.8**	**72.5**	**72.1**	**71.9**
	ViT	sam-sgd	**9.9**	**9.8**	**10.2**	66.0	63.7	60.8
	ViT	sgd	8.0	9.5	9.3	**66.5**	**65.2**	**62.9**
MNIST	ResNet18	adam	**16.3**	**9.9**	**10.3**	**99.4**	99.5	99.4
	ResNet18	sam-adam	9.7	9.7	9.7	99.3	**99.5**	**99.5**
	ResNet18	sam-sgd	11.3	**11.3**	**11.3**	97.4	97.0	96.5
	ResNet18	sgd	**11.5**	**11.3**	**11.3**	**97.7**	**97.5**	**97.0**
	ViT	adam	16.7	12.8	22.7	97.0	96.6	95.7
	ViT	sam-adam	**44.6**	**19.1**	**25.7**	**97.6**	**97.3**	**96.8**
	ViT	sam-sgd	**13.8**	12.3	10.7	85.2	79.1	75.7
	ViT	sgd	12.2	**21.8**	**12.0**	**87.6**	**83.9**	**77.8**

TABLE I: Classification accuracy and adversarial accuracy for the different optimizers and architecture configurations under different noisy training levels, P, on CIFAR-10 and MNIST datasets. All SAM results are reported for $\rho = 0.05$. The best result for each configuration is highlighted in bold.

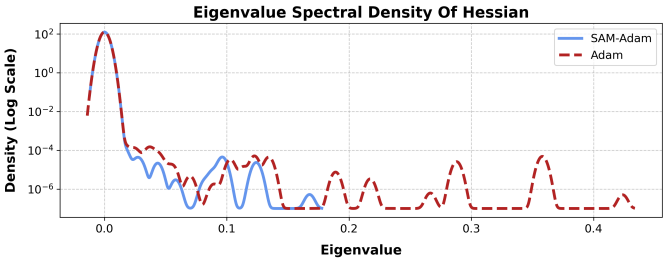

Fig. 1: Estimated Eigenvalue spectral density (ESD) of the Hessian matrix for ResNet18 trained on CIFAR10 with Adam and SAM-Adam optimizer.

score-based model is trained for 150k steps (LR 2×10^{-4}), the DDPM for 29k steps (LR 10^{-3}), and flow matching for 200k steps (LR 2×10^{-4}). Full configurations for these models are provided in Table V-VII.

The models are evaluated using Fréchet Inception Distance (FID) based on 6000 samples [18] and sample quality. To assess robustness, we perturb model weights with Gaussian noise and track the degradation in FID. We hypothesize that SAM-trained models, due to flatter minima, exhibit greater resilience to such perturbations.

IV. Results

We present the results of our experiments in two main sections: classification IV-A and generative modeling IV-B.

A. Classification Results

In Table I, we summarize classification accuracy across optimizers, datasets, and architectures under varying levels of label noise. For $P = 0$ (no label noise), accuracy differences between SAM and its base optimizer are minor, except for ResNet18 on CIFAR-10, where SAM-SGD yields a notable improvement. Under noisy training ($P \in 0.1, 0.2$) the results are mixed. SAM-SGD often underperforms compared to SGD, while SAM-Adam under noisy training consistently improves over Adam, especially for ViT.

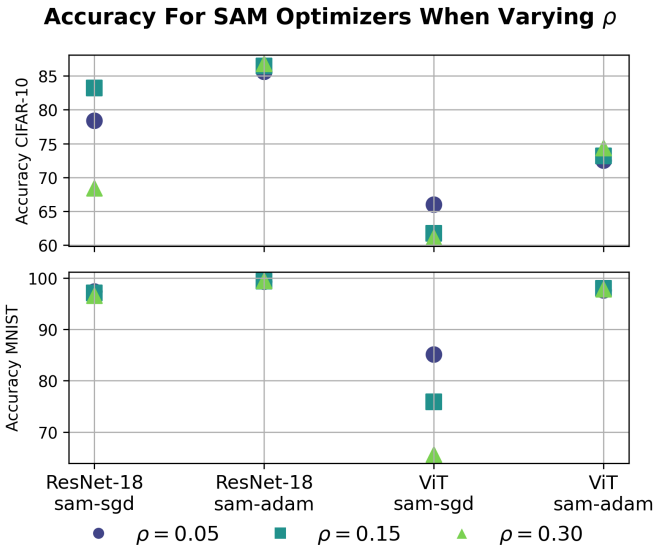

Fig. 2: Accuracy under varying ρ and noise probabilities.

A similar pattern is observed for adversarial robustness. SAM-Adam achieves significant gains on ViT (especially on MNIST), while SAM-SGD shows less consistent improvements. Hence, the expected behaviour of SAM being more robust under noisy training and adversarial attacks is not always observed. SAM's benefits appear more pronounced in some configurations, particularly when paired with Adam and transformer-based models.

To investigate why SAM-Adam outperforms Adam for a specific configuration, we examine the loss curvature for ResNet18 (P=0) trained on CIFAR10 by estimating the eigenvalue spectral density of the hessian (Figure 1). Both spectra are centered near zero, but Adam exhibits a longer right tail, indicating more sharp directions, whereas the directions for SAM-Adam appears more flat. Both optimizers also show small negative eigenvalues, implying that the optimizers have not yet converged to a local minimum satisfying the first and second order optimality conditions fully. In Appendix VI-B, Figure 6 we show the loss landscape in the direction of the largest and second largest eigenvector of the hessian, which echoes the above finding that the loss landscape for SAM-Adam is flatter than for Adam.

Finally, in Figure 2, we analyze the effect of varying the SAM hyperparameter ρ on classification accuracy under different noise levels. We observe that in general $\rho = 0.05$ seems to be a good choice and for the most part only tiny variations are observed for different ρ values for these specific experiments.

B. Generative Modeling Results

In Table II, we present the FID scores for the three different generative models (score-based diffusion, DDPM, and flow matching) trained with Adam and SAM optimizers under varying levels of Gaussian noise ϵ added to the model parameters. We observe that SAM consistently outperforms Adam across

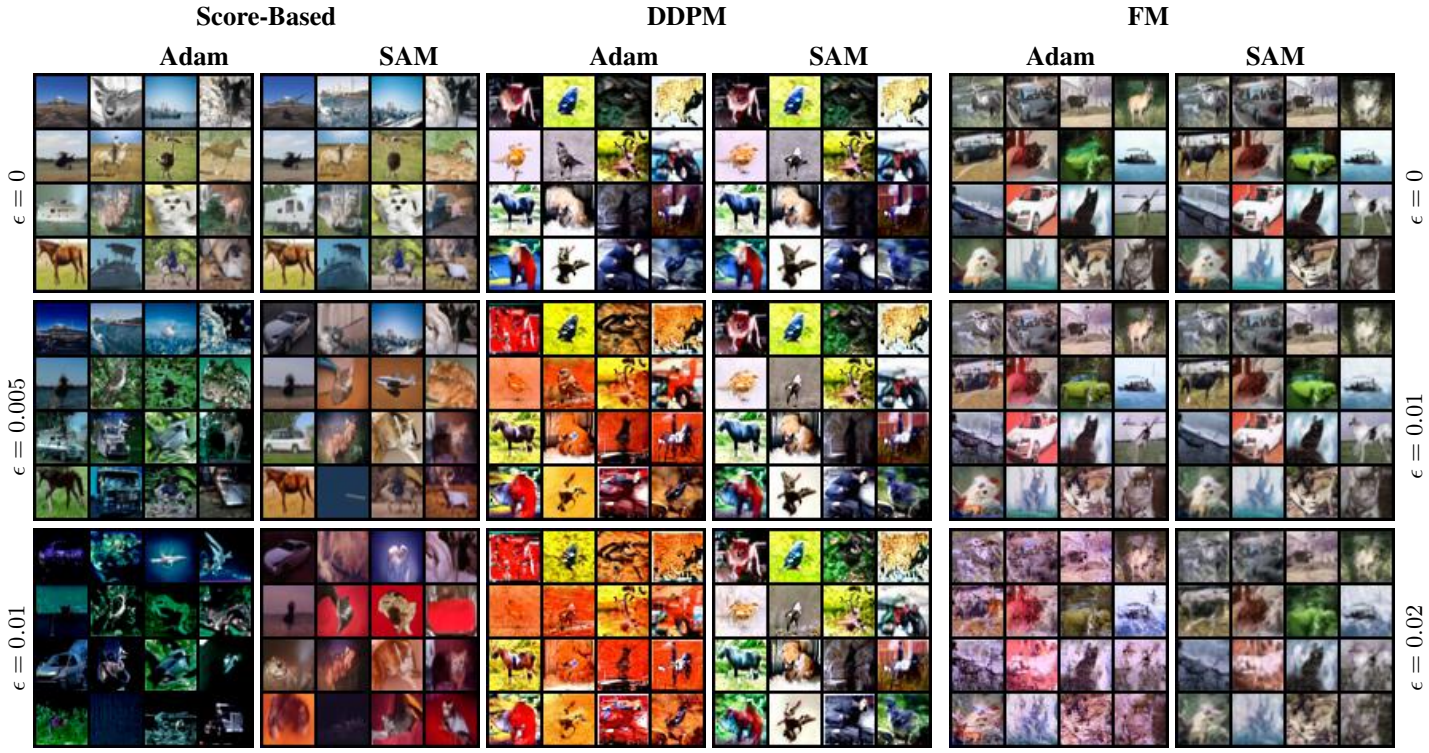

Fig. 3: Generated CIFAR-10 samples under increasing parameter perturbation. The left block shows Score-based and DDPM models with shared ϵ values; Flow Matching (right) uses a separate noise scale. SAM generally improves sample robustness under noise.

	FID Scores ↓						
	Score-Based		**DDPM**			**FM**	
ϵ	Adam	SAM	Adam	SAM	ϵ	Adam	SAM
0	**24.0**	24.9	**24.3**	24.9	0	**12.3**	15.3
0.001	27.2	**25.4**	28.3	**25.2**	0.005	**14.5**	18.2
0.005	92.0	**43.7**	84.8	**28.1**	0.01	**27.7**	30.6
0.01	208.8	**112.9**	156.7	**44.7**	0.02	163.0	**142.9**

TABLE II: Fréchet Inception Distance (FID) scores for different generative image models and optimizers (Adam,SAM) under varying levels of Gaussian noise ϵ added to the model parameters. Lower FID indicates better quality. SAM generally improves sample robustness under noise.

all noise levels for both score-based and DDPM models, with particularly large gains at higher noise levels ($\epsilon \geq 0.005$).

For the flow-matching model we generally observed that higher noise-level was needed in order to perturb the model weights to generate less coherent images. While the differences between SAM and Adam are less pronounced for flow matching, SAM again shows greater robustness at higher noise ($\epsilon = 0.02$), maintaining lower FID scores.

In Figure 3, we visualize generated samples under different noise perturbation levels. The SAM trained models retain higher visual fidelity and diversity than their Adam trained counterparts, especially as noise increases. These findings could support the hypothesis that SAM finds flatter minima which in turn leads to more robust generative models, preserving sample quality even under significant weight perturbations.

V. Discussion & Conclusion

In this project, we investigated the effectiveness of SAM across various tasks, including image classification and generative modeling. Our findings for classification are rather mixed, but we find that SAM-Adam in conjunction with ViT architectures consistently outperforms Adam, especially under noisy training conditions. Curvature analysis for a particular configuration further supported the hypothesis that SAM encourages convergence to flatter minima.

For generative modeling, SAM consistently outperformed Adam across all tested noise levels for score-based diffusion and DDPM models, and also showed improved robustness for flow matching at higher perturbation levels. This suggests that SAM's ability to potentially find flatter minima leads to more robust generative models that maintain sample quality even when perturbed. This promising result is to our knowledge the first evaluation of SAM in this setting and should be studied further.

While SAM introduces additional computational overhead due to its two-step optimization process, the observed improvements in generalization and robustness may justify the added cost, especially in settings where model reliability is critical. The findings of this work motivate further exploration of SAM in more domains and architectures, especially within generative modeling, where early results appear promising.

REFERENCES

[1] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[2] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," 2021. [Online]. Available: https://arxiv.org/abs/2010.01412

[3] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," 2019. [Online]. Available: https://arxiv.org/abs/1903.12261

[4] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, p. 665–673, Nov. 2020. [Online]. Available: http://dx.doi.org/10.1038/s42256-020-00257-z

[5] P. Mi, L. Shen, T. Ren, Y. Zhou, X. Sun, R. Ji, and D. Tao, "Make sharpness-aware minimization stronger: A sparsified perturbation approach," 2022. [Online]. Available: https://arxiv.org/abs/2210.05177

[6] M. Andriushchenko and N. Flammarion, "Towards understanding sharpness-aware minimization," 2022. [Online]. Available: https://arxiv.org/abs/2206.06232

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1512.03385

[8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2021. [Online]. Available: https://arxiv.org/abs/2011.13456

[10] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020. [Online]. Available: https://arxiv.org/abs/2006.11239

[11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," 2023. [Online]. Available: https://arxiv.org/abs/2210.02747

[12] L. Deng, "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[13] A. Krizhevsky, "Learning multiple layers of features from tiny images," pp. 32–33, 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[14] E. Wong, L. Rice, and J. Z. Kolter, "Fast is better than free: Revisiting adversarial training," 2020. [Online]. Available: https://arxiv.org/abs/2001.03994

[15] S. Ubaru, J. Chen, and Y. Saad, "Fast estimation of $tr(f(a))$ via stochastic lanczos quadrature," *SIAM Journal on Matrix Analysis and Applications*, vol. 38, no. 4, pp. 1075–1099, 2017. [Online]. Available: https://doi.org/10.1137/16M1104974

[16] Z. Yao, A. Gholami, K. Keutzer, and M. Mahoney, "Pyhessian: Neural networks through the lens of the hessian," 2020. [Online]. Available: https://arxiv.org/abs/1912.07145

[17] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," 2015. [Online]. Available: https://arxiv.org/abs/1505.04597

[18] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," 2018. [Online]. Available: https://arxiv.org/abs/1706.08500

VI. APPENDIX

A. Hyperparameters for Classification Experiments

The hyperparameters used for the classification experiments are summarized in Table III.

Dataset	Base Optimizer	Architecture	Learning Rate	Batch Size
MNIST,CIFAR10	SGD	ResNet18,ViT	0.01	256
MNIST,CIFAR10	Adam	ResNet18	0.001	256
MNIST,CIFAR10	Adam	ViT	0.0001	256

TABLE III: Hyperparameters for classification experiments across different datasets and architectures.

For both SGD and Adam we use the default settings for the PyTorch implementations. These settings are summarized in Table IV.

Optimizer	Parameter	Value
SGD	Momentum	0
SGD	Weight Decay	0
Adam	Beta1	0.9
Adam	Beta2	0.999
Adam	Epsilon	10^{-8}
Adam	Weight Decay	0

TABLE IV: Hyperparameters for SGD and Adam used in classification experiments.

B. Results: Classification

In Figure 4 and Figure 5 we visualize the results also presented in Table I.

C. Hyperparameters Generative Modeling

DDPM: In Figure V we present the key hyperparameters used for the DDPM experiments.

Parameter	Value
Batch Size	256
Epochs	150
Training Steps	29400
ρ	0.05 (Only SAM)
Lr	10^{-3}
Weight Decay	0
Sampling Steps	1000
Model	Transformer U-Net
Model Params	30 M

TABLE V: Hyperparameters for DDPM diffusion experiments.

Score Based Diffusion: In Figure VI we present the key hyperparameters used for the Score-based diffusion experiments.

Parameter	Value
Batch Size	128
Epochs	400
Training Steps	150,000
ρ	0.05 (Only SAM)
Lr	$2 \cdot 10^{-4}$
Weight Decay	0
Sampling Steps	1000
Model	Transformer U-Net
Model Params	63 M

TABLE VI: Hyperparameters for Score-based diffusion experiments.

Flow Matching: In Figure VII we present the key hyperparameters used for the flow matching experiments.

Parameter	Value
Batch Size	128
Training Steps	200,000
ρ	0.05 (Only SAM)
Learning Rate	$2 \cdot 10^{-4}$
Weight Decay	0
#Integration Steps	100
Integration Method	dopri5
Model	Transformer U-Net
#Model Params	35 M

TABLE VII: Hyperparameters for flow matching experiments.

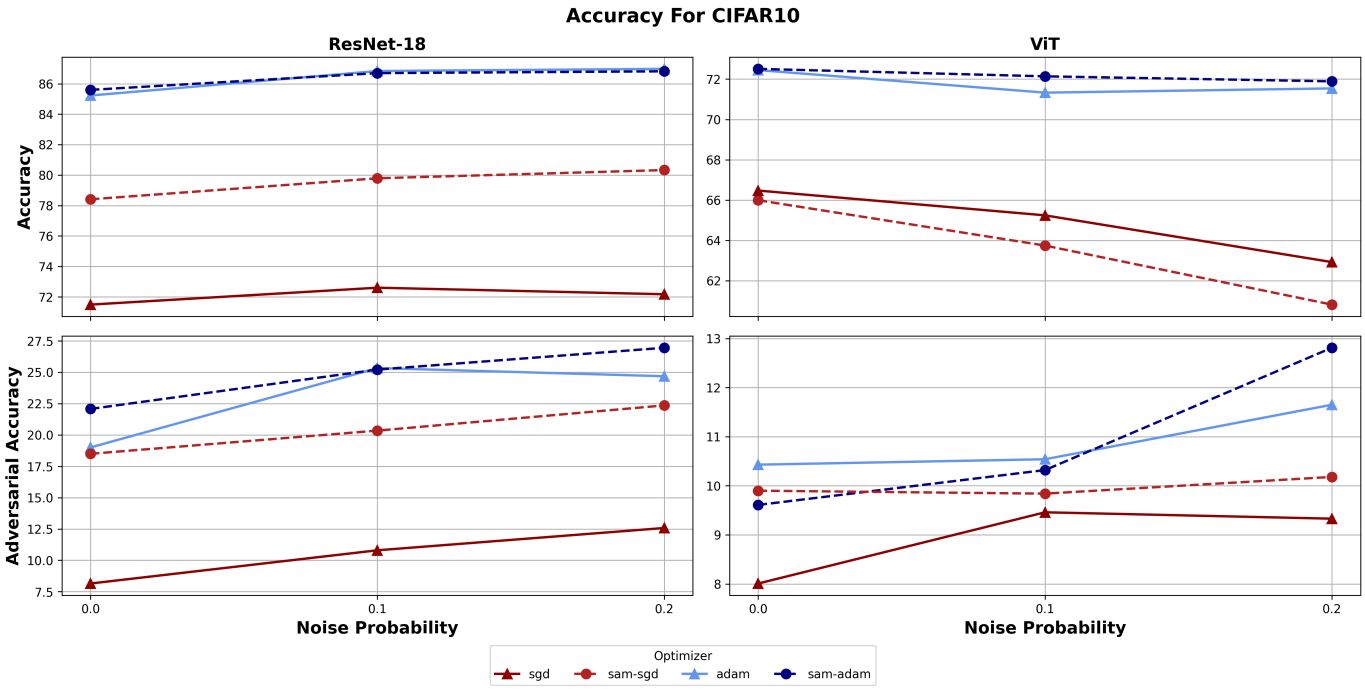

Fig. 4: Classification results for CIFAR10 for ResNet18 and ViT trained using either sgd, sam-sgd, adam or sam-adam.

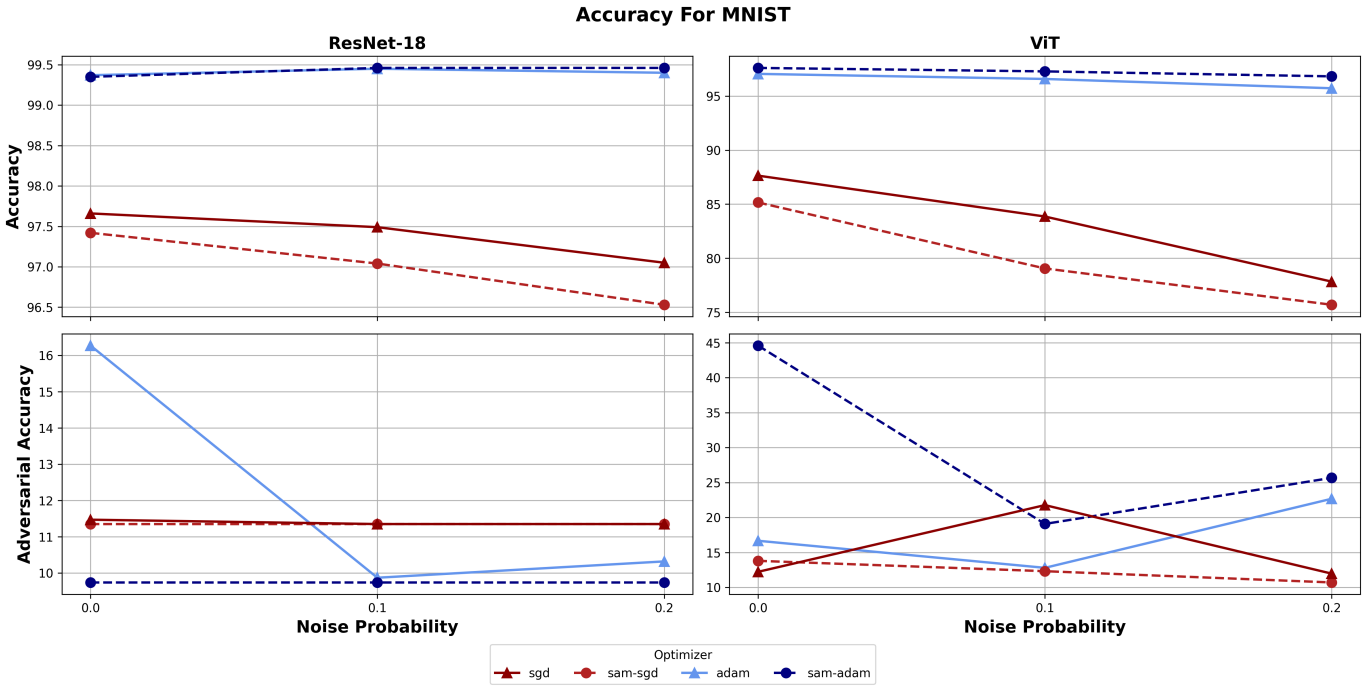

Fig. 5: Classification results for MNIST for ResNet18 and ViT trained using either sgd, sam-sgd, adam or sam-adam.

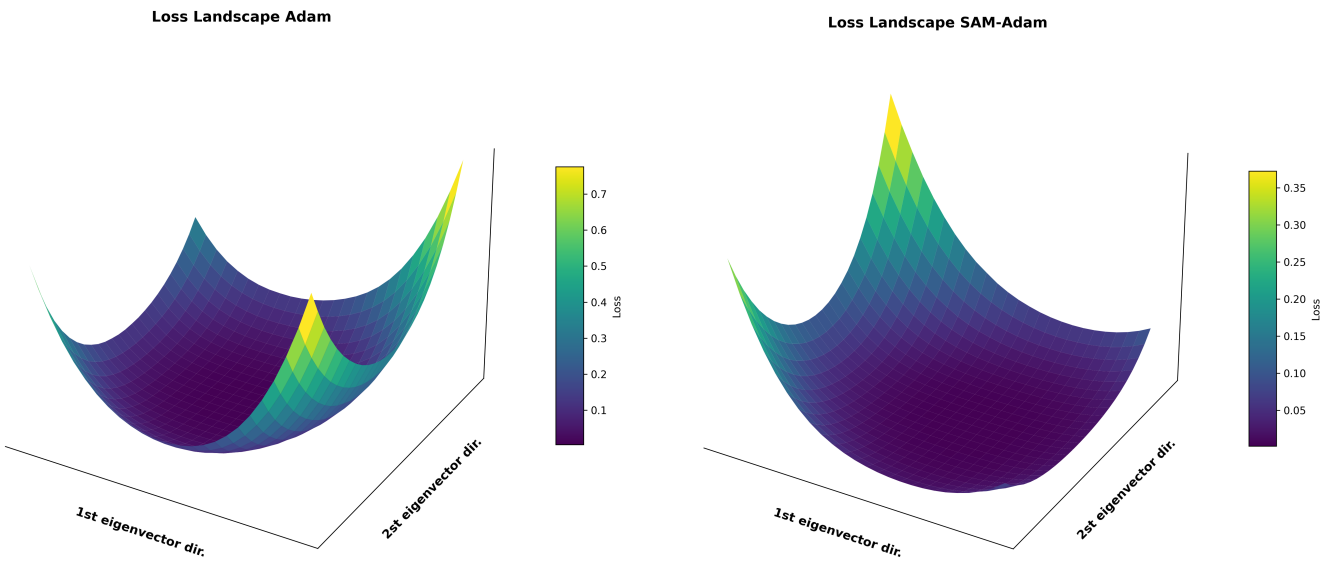

Fig. 6: Comparison of loss landscape in the direction of the top-2 eigenvectors associated with the largest eigenvalues of the hessian for a ResNet-18 trained on CIFAR-10 with Adam and SAM optimizers in a none noisy setting. We observe the loss curvature for SAM is more flat than it is for Adam.